JalSetu

Project Documentation: Setup and Execution Guide

Backwater Cargo Exchange: shipment-first decisions for Kerala's inland waterways

IBM x Kerala Government Hackathon 2026 | Challenge 7 | Team Vortex

Document date: 7 October 2026

Verified by: fresh clone of the repository, install, test run and a browser walkthrough on Windows 11

HOW TO READ THIS This guide is self-contained. It takes you from an empty machine to a running prototype in about ten minutes of work, then walks you through every screen. Everything here was checked against the running system; anything not built is marked **ROADMAP**.

Contents

1 Overview and what you are setting up

2 Prerequisites

3 Get the code

4 Backend: install and run

5 Frontend: install and run

6 Run the automated tests

7 Guided walkthrough of the application

8 Roles and screens reference

9 Project structure

10 Configuration and data

11 API reference

12 Optional tasks

13 Troubleshooting

14 Limitations and what is real

15 Third-party tools, AI tools and licences

A Command cheat sheet and tested environment

1. Overview and what you are setting up

JalSetu is a prototype that helps a cargo owner in Kerala decide how to move a shipment: by road, by water (a hybrid plan with a barge leg between jetties), or by road alone when road is better. It compares the options on cost, time, CO_2 and reliability, recommends one with a plain-English reason, matches the shipment to vessel sailings, and keeps a person in charge of every booking, re-plan and hazardous-cargo review. Boat owners, warehouses and planners use the same platform.

It was built spec-first with **IBM Bob**, IBM's AI coding agent, as our development partner.

It has two parts that you start separately:

Part	What it is	Address
Backend	A FastAPI server (Python). It holds a deterministic decision engine, in-memory data and the API.	`http://localhost:8000`
Frontend	A React + Vite web app with four role views: Shipper, Boat owner, Warehouse, Planner.	`http://localhost:5173`

Setup at a glance (details in sections 4 and 5)

```
# terminal 1 (repository root)
git clone https://github.com/gop-i-krishnan/keralai-team-vortex.git
cd keralai-team-vortex
python -m venv .venv                  # then activate it (section 4)
pip install -r backend/requirements.txt
python -m uvicorn backend.api.main:app --port 8000

# terminal 2
cd frontend
npm install
npm run dev                           # then open http://localhost:5173
```

Three things to know before you start

- **No API keys, no internet at run time.** The demo is fully offline. Internet is needed only to download packages during installation.
- **State is in memory.** Restarting the backend, or pressing *Reset Demo* in the app, restores the seed data. Nothing is saved to disk.
- **Honest labels.** Numbers carry a `[sourced]`, `[assumed]` or `[simulated]` tag. Most rates are assumptions, and tracking, verification and notifications are simulated. See section 14.

2. Prerequisites

Requirement	Tested version	Notes
Python	3.14.5	Other versions are untested. Check with `python --version`.
pip	26.1.1	Comes with Python.

Node.js and npm	Node 24.14.0	Check with `node --version`. A recent Node is needed for Vite 8.
Git	any recent	Only needed to clone the repository.
Operating system	Windows 11	Tested in PowerShell and Git Bash. macOS and Linux commands are given but were not tested.
Web browser	Chrome / Edge	Any modern browser should work. The screenshots here came from headless Chrome at 1280 px width.
Access to the repository	a GitHub account	The repository is private. You need to be added as a collaborator to clone it.

Disk space and RAM needs are small; we did not measure them.

3. Get the code

```
git clone https://github.com/gop-i-krishnan/keralai-team-vortex.git
cd keralai-team-vortex
```

All later commands assume your terminal is in this **repository root** (the folder containing `backend`, `frontend`, `data` and `docs`) unless stated otherwise. The folder name itself does not affect how the project runs.

4. Backend: install and run

4.1 Create and activate a virtual environment

Shell	Commands
Windows PowerShell	`python -m venv .venv` `.venv\Scripts\Activate.ps1`
Git Bash (Windows)	`python -m venv .venv` `source .venv/Scripts/activate`
macOS / Linux (untested)	`python3 -m venv .venv` `source .venv/bin/activate`

> **IF POWERSHELL BLOCKS ACTIVATION** with "running scripts is disabled", run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in the same window, then activate again. This only affects that window.

4.2 Install dependencies

```
pip install -r backend/requirements.txt
```

The file pins exact versions of eight packages: fastapi 0.142.2, uvicorn[standard] 0.54.0, pydantic 2.13.5, pyyaml 6.0.3, rapidfuzz 3.14.6, pytest 9.1.1, httpx 0.28.1 and python-dotenv 1.2.4. The optional `ibm-watsonx-ai` package is deliberately not installed (section 12).

IF INSTALL FAILS with "No matching distribution found for rapidfuzz==3.14.6", it was a pip cache problem on our test machine. Run `pip install --no-cache-dir -r backend/requirements.txt`.

4.3 Start the server

Shell	Command (from the repository root)
Git Bash / macOS / Linux	`JALSETU_MOCK_LLM=1 python -m uvicorn backend.api.main:app --port 8000`
Windows PowerShell	`$env:JALSETU_MOCK_LLM="1"; python -m uvicorn backend.api.main:app --port 8000`

Leave this terminal open. You should see Uvicorn report that it is running on `http://127.0.0.1:8000`.

4.4 Check that it works

In a second terminal, or in your browser:

```
curl http://localhost:8000/
# expected: {"status":"ok","service":"JalSetu"}
```

Interactive API documentation (Swagger) is at `http://localhost:8000/docs`.

GOOD TO KNOW

- `JALSETU_MOCK_LLM=1` selects the offline parser. If the variable is not set at all, the backend still defaults to this offline mode.
- The backend does **not** read a `.env` file. Set variables in your shell as shown. The `.env.example` file documents the variable names with dummy values only.
- Use **port 8000**: the frontend expects it by default. Run the command from the repository root so that `backend.api.main` resolves.

5. Frontend: install and run

Open a **second terminal** (the backend keeps running in the first).

```
cd frontend
npm install
npm run dev
```

Vite prints a local address. Open **`http://localhost:5173`** in your browser. You should land on the Shipper view with the deadline slider.

PORTS MATTER The backend allows browser requests only from `http://localhost:5173` and `http://127.0.0.1:5173`. If you run Vite on another port the app will show "Cannot reach the JalSetu server". To use a different backend address, create `frontend/.env` containing `VITE_API_URL=http://localhost:8000/api` (adjust the address) and restart `npm run dev`. A different frontend port also requires changing the allowed origins in `backend/api/main.py`.

Other frontend commands

Command	Purpose

`npm run build`	Production build into `frontend/dist` (git-ignored). Verifies the code compiles.
`npm run lint`	Runs oxlint. Currently reports 10 warnings and 0 errors.
`npm run preview`	Serves the built files. Note it uses port 4173, which the backend does not allow; use `npm run dev` for the demo.

6. Run the automated tests

With the virtual environment active, from the repository root:

```
python -m pytest backend/ -q
```

Expected result: `261 passed` in a few seconds (one third-party deprecation warning from the test client is normal). No server needs to be running, and no network is used.

Area	Test files (count)
Decision engine	options (13), matching (15), reliability (7), viability (5), parser (11)
Safety and hazard rules	hazards (10), warehouse (13)
Vessels and history	registry (34), history (11)
Disruption and tracking	replan (31), tracking (14)
API (end to end)	api (17), vessel (26), replan (25), warehouse (19), compliance (6)
Documentation in sync	docs sync (4): fails if the data tables in `docs/SPEC.md` drift from `data/config.yaml` and `data/seed.json`

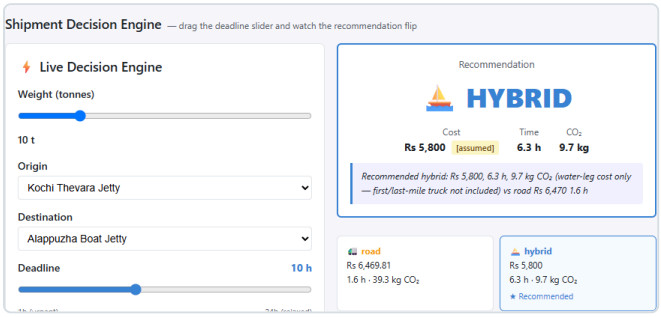
Frontend behaviour has **no automated tests**. It was verified by driving the real app in a browser (section 7).

7. Guided walkthrough of the application

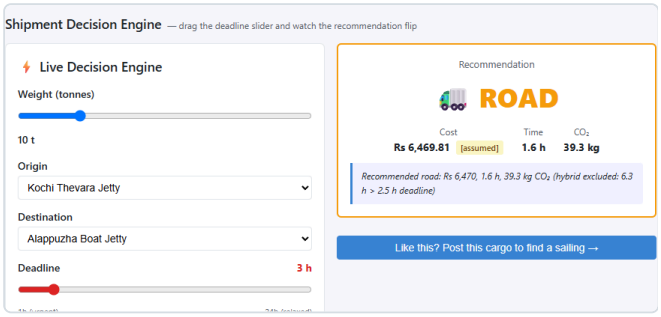
With both servers running, follow these steps. Each ends with what you should see. Total time: about five minutes.

Step 1. The decision (landing screen, no clicks)

The Shipper view opens on **Compare options**. The shipment is 10 t from Kochi Thevara Jetty to Alappuzha Boat Jetty. Drag the **Deadline** slider.



Deadline 10 h: **HYBRID**, Rs 5,800, 6.3 h, 9.7 kg CO_2.

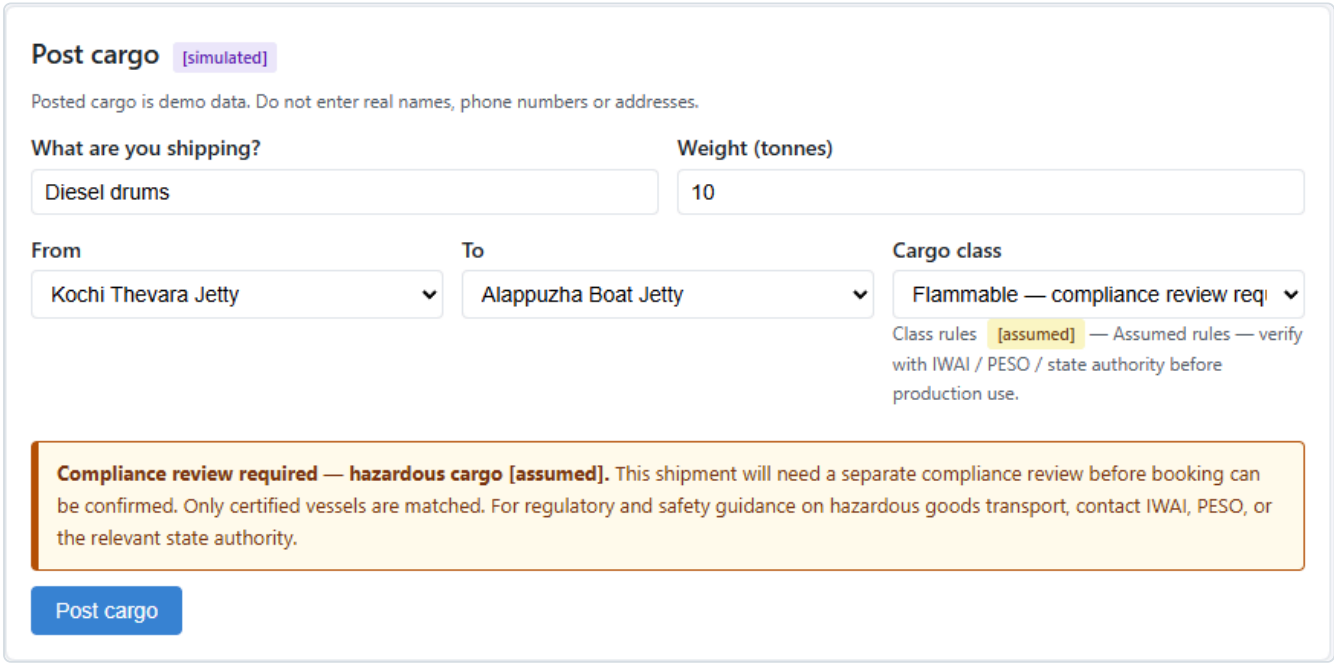


Deadline 3 h: **ROAD**, Rs 6,470, 1.6 h, 39.3 kg CO_2.

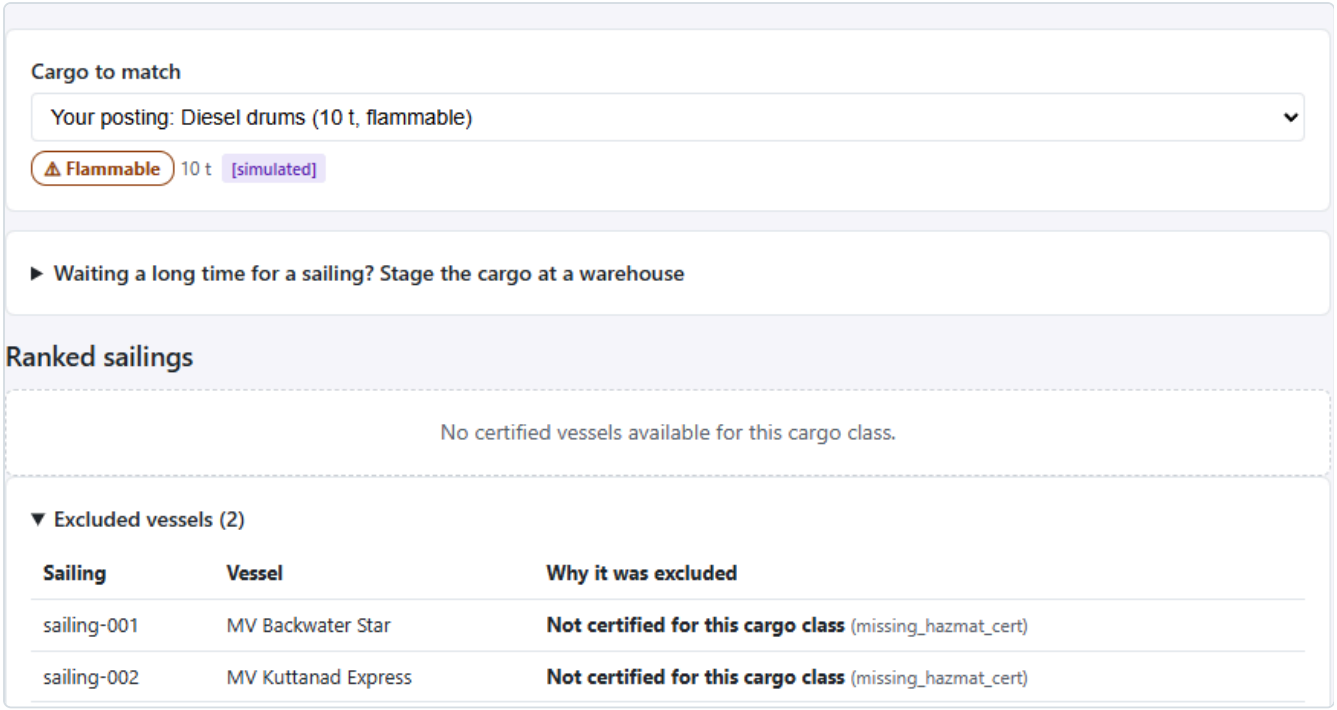
> **READ THIS CAVEAT** The hybrid cost covers the barge leg and jetty handling only. It excludes first and last-mile trucking, whereas road is door to door. So the roughly 10% cost and 75% CO_2 advantage is for the barge leg, not a door-to-door claim. All rates are assumptions.

Step 2. A hazardous cargo is blocked, with reasons

1. Click the **Post cargo** tab. Type a description, and choose cargo class **Flammable**. A "Compliance review required" banner appears.
2. Click **Post cargo**, then **Find matching sailings**.



Choosing a hazardous class shows a banner with a text label (colour is never the only signal).



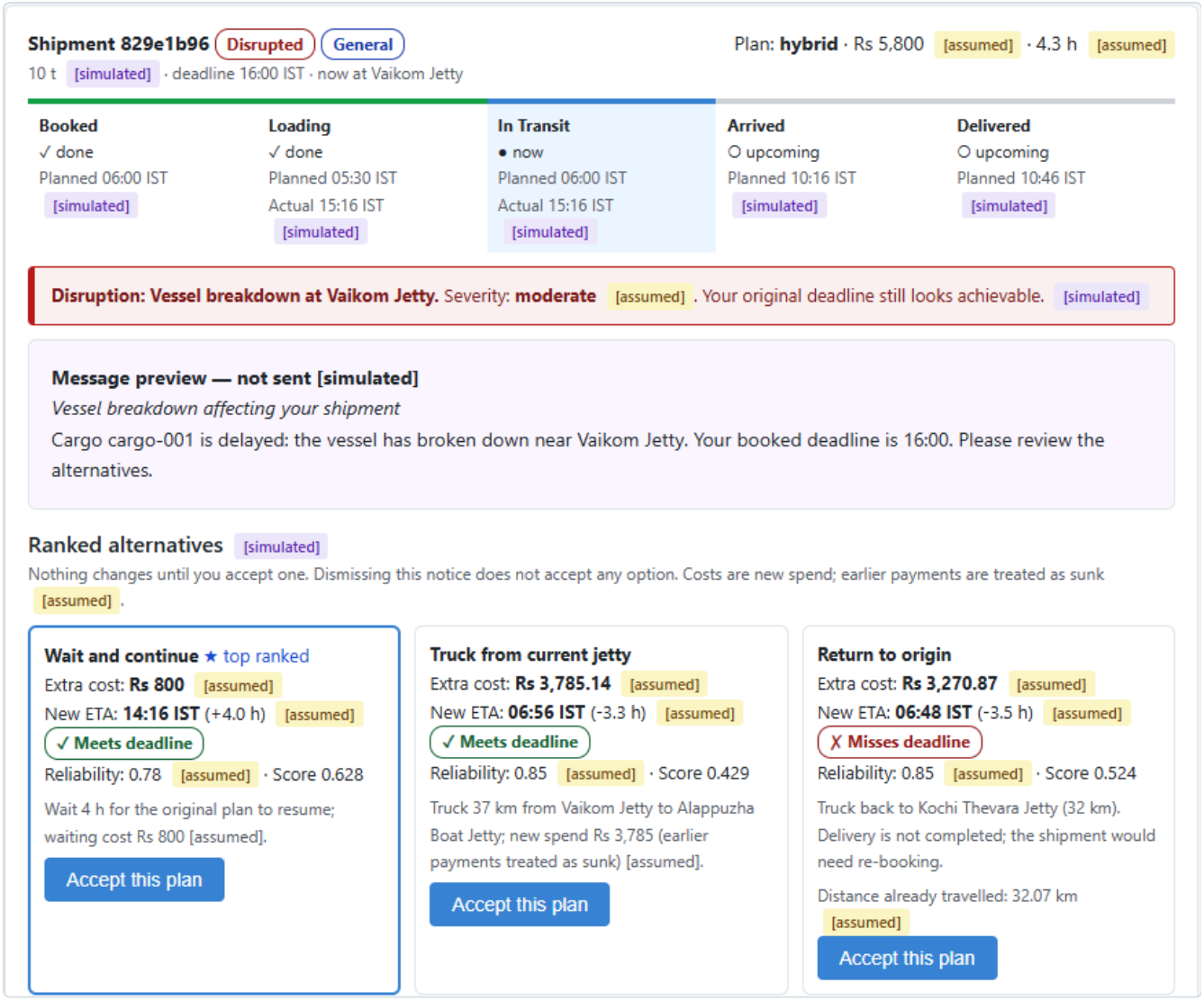
Expected: "No certified vessels available for this cargo class", with both sailings listed under *Excluded vessels* and the reason `missing_hazmat_cert`.

Step 3. Book a cargo with a two-step confirmation

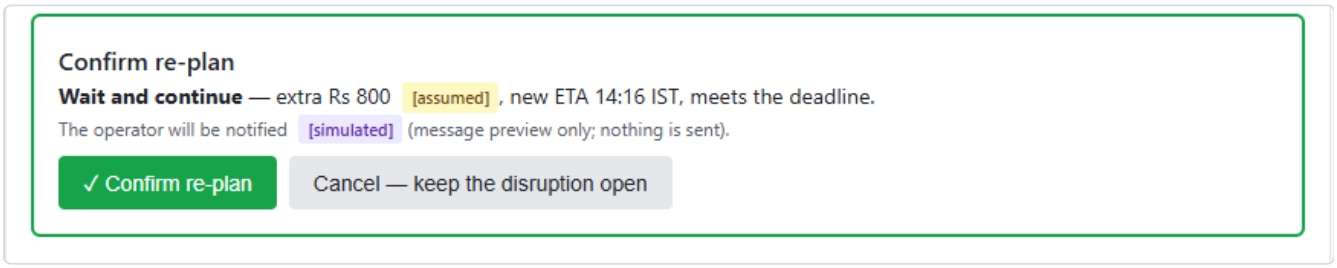
1. Back in **Post cargo**, post a **General** cargo and click **Find matching sailings**.
2. Click **Select and continue to Confirm** on a sailing, then **Preview booking**.
3. Click **Confirm booking**. Expected: "Booking confirmed". Nothing is booked until this click.

Step 4. A disruption, handled by a person

1. Go to **My shipments**. Click **Advance milestone (demo)** twice. The shipment is now In Transit, at Vaikom Jetty.
2. Click **Simulate disruption**, then **Trigger disruption**. A banner, a "Message preview, not sent" card and a deadline note appear.
3. Click **Show alternatives**: ranked options such as *Wait and continue*, *Truck from current jetty* and *Return to origin*, each with extra cost, new ETA and whether the deadline is met.
4. Click **Accept this plan** on one, then **Confirm re-plan**. Expected: "Re-plan confirmed". Until that last click the booking is unchanged.



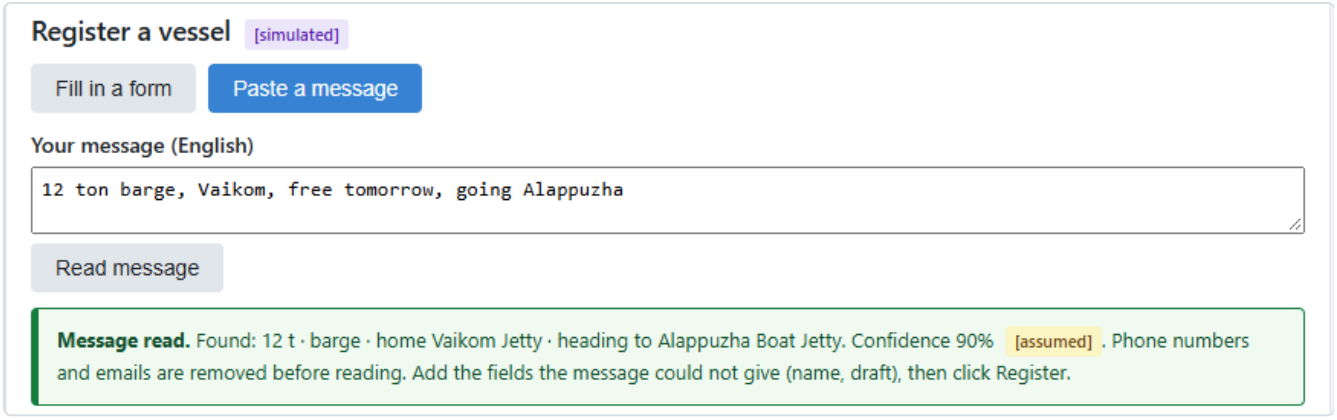
A simulated vessel breakdown, with the notification preview and ranked alternatives.



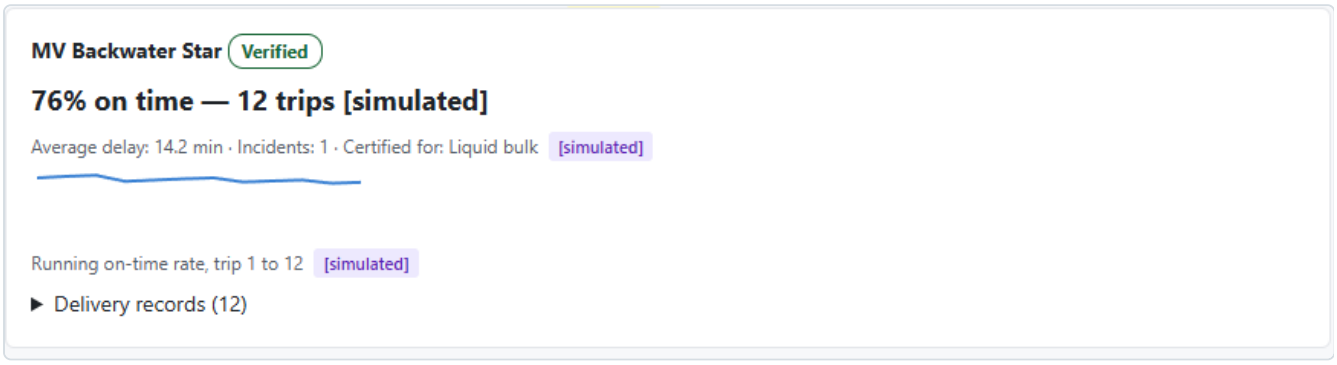
After "Accept this plan", the explicit "Confirm re-plan" dialog appears. Cancelling keeps the disruption open and accepts nothing.

Step 5. The boat owner side

1. Click **Boat owner** then **Register vessel**. Choose **Paste a message** and click **Read message** (the example text is pre-filled).
2. The capacity, vessel type and home jetty fill in. Add a vessel name and a draft (the message cannot supply them), then click **Register vessel**.
3. Open **My sailings**: click **Mark Verified (demo admin)** and publish a recurring sailing. Open **My history** to see on-time records with trip counts.



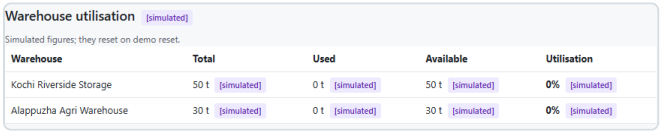
Message reading is a simple pattern reader (English only). Nothing is registered until you click Register.



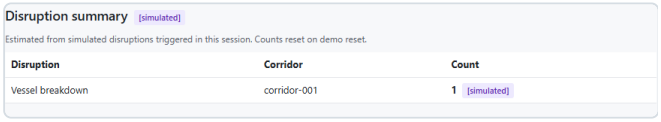
"76% on time, 12 trips [simulated]": a percentage is never shown without its trip count.

Step 6. Warehouse and planner views

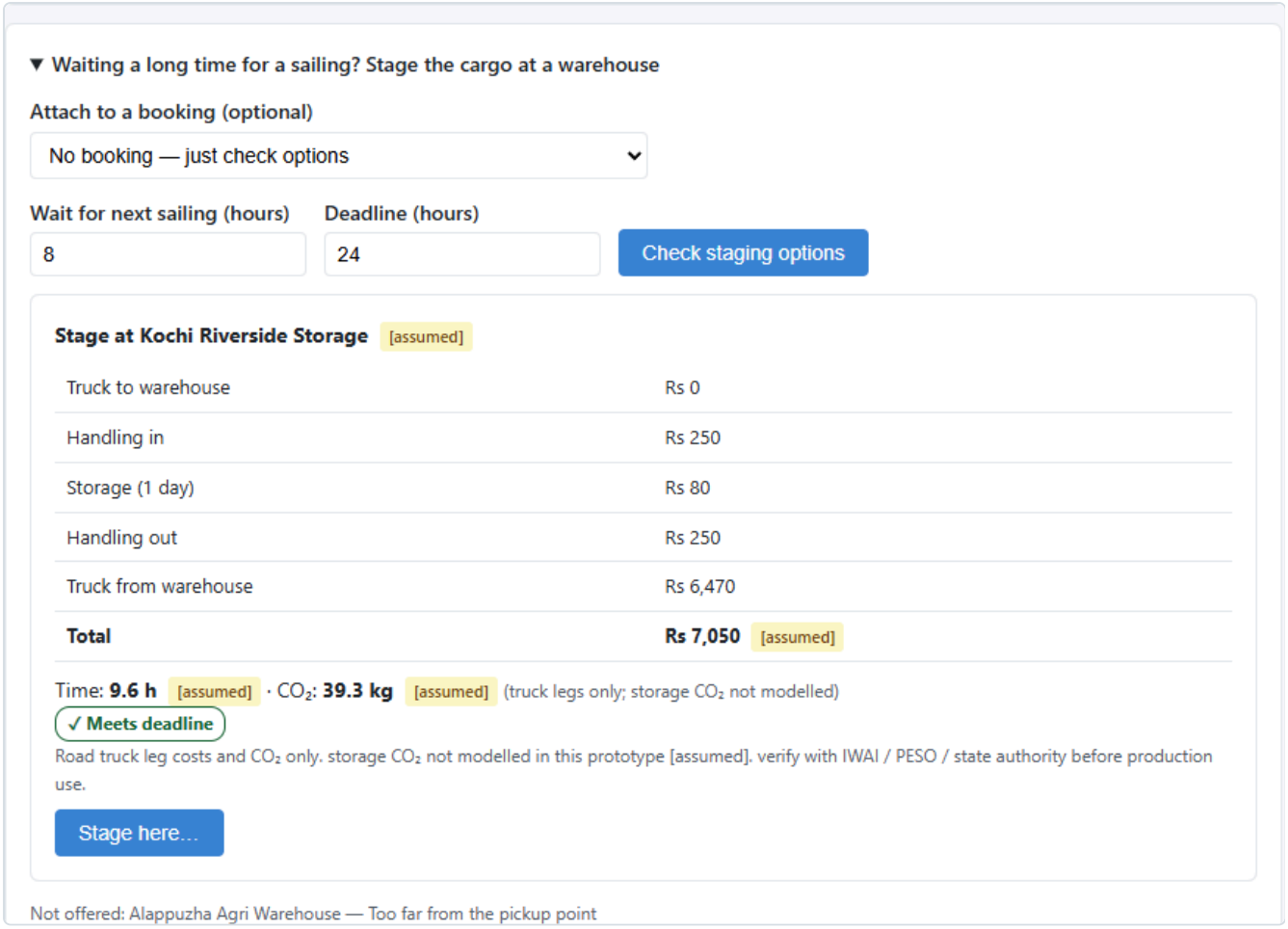
- **Warehouse** → **Utilisation** shows capacity used and available. **Register warehouse** adds one.
- In **Shipper** → **Matches**, open "Waiting a long time for a sailing?" and click **Check staging options** for an itemised warehouse plan. Staging can be attached to a booking.
- **Planner** → **Corridor viability** and **Disruption summary**.



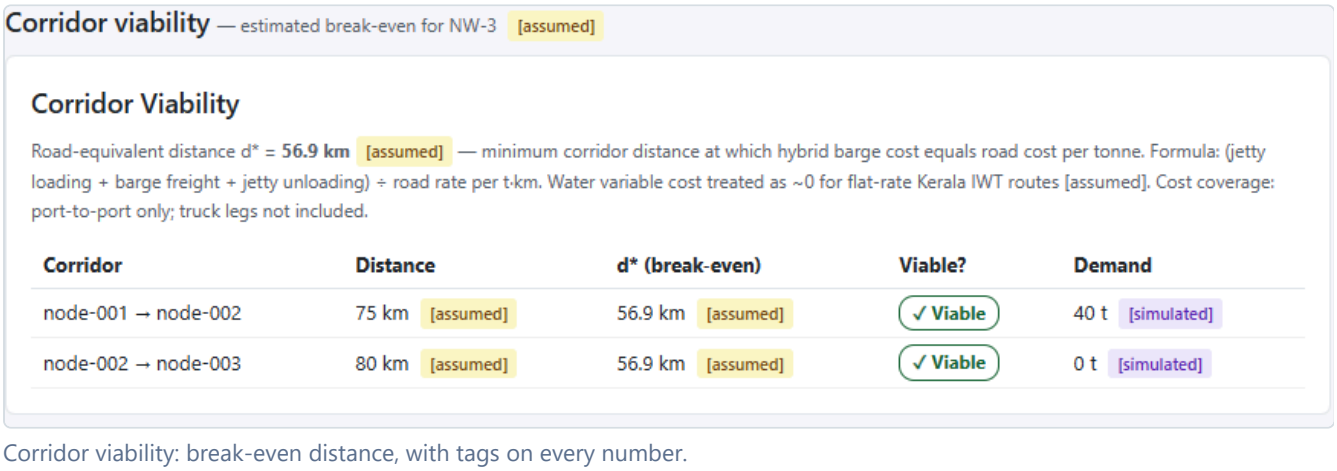
Warehouse utilisation (simulated data).



Disruption summary by type and corridor.



An itemised staging plan: five cost lines, time, CO_2 with its scope, and a deadline check. "Stage here" opens a confirmation.



Corridor viability: break-even distance, with tags on every number.

Step 7. Reset

Click the red **Reset Demo** button (top right) at any time. It restores the seed data and returns you to the landing screen.

8. Roles and screens reference

Role	Screen	What it does

Shipper	Compare options	Road vs hybrid comparison with a reason; deadline slider; subsidy what-if (off by default).
	Post cargo	Create a cargo with weight, route, hazard class; plain-language chat (simulated, English only).
	Matches	Ranked sailings; excluded vessels with reasons; optional warehouse staging.
	Confirm	Two-step booking; compliance review for hazardous cargo; backhaul and pooling counts.
	My shipments	Milestone timeline (simulated), disruption demo control, alternatives, re-plan confirm, mark delivered, event log.
Boat owner	Register vessel	By form or pasted message. Per-field errors; duplicate guard.
	My sailings	Vessels with status, demo verify / suspend / reinstate, publish sailings.
	My history	On-time record and delivery records, with trip counts.
Warehouse	Register warehouse	Capacity, costs, cargo classes accepted.
	Utilisation	Capacity used and available (simulated).
Planner	Corridor viability	Break-even distance and static corridor map.
	Disruption summary	Counts by disruption type and corridor (simulated).

There is no login. The Boat owner view has a "Viewing as" selector to switch between the two fictional operators.

9. Project structure

```
repository root
├── backend/
│   ├── requirements.txt          pinned Python dependencies
│   ├── data_store.py             in-memory state, reset to seed
│   ├── engine/                   PURE decision logic (no I/O, no LLM calls)
│   │   ├── options.py            road vs hybrid plans and scoring
│   │   ├── matching.py           sailing ranking and exclusions
│   │   ├── hazards.py            hazard hard filters
│   │   ├── registry.py history.py   vessel lifecycle, delivery history, reliability
│   │   ├── replan.py tracking.py    disruption alternatives, milestones
│   │   ├── warehouse.py geo.py parser.py reliability.py viability.py ...
│   │   └── tests/                engine unit tests
│   └── api/
│       ├── main.py               FastAPI app and CORS
│       ├── models.py             request/response models
│       ├── routers/              one file per endpoint group
│       ├── llm/                  offline mock adapter; optional IBM watsonx adapter
│       └── tests/                API tests
├── frontend/
│   ├── package.json
│   └── src/ (App.jsx, api.js, index.css, components/)
├── data/
│   ├── config.yaml               every constant, tagged sourced / assumed
│   └── seed.json                 fictional demo data, tagged simulated
├── docs/                         SPEC.md, SUBMISSION.md, disclosures.md, screenshots/,
│                                 sync_tables.py, the presentation, this guide
├── specs/                        feature specs 001-006 (spec, plan, tasks, contracts)
├── .specify/                     constitution and Spec Kit files
├── README.md  ATTRIBUTIONS.md  .env.example
```

Design rule worth knowing: `backend/engine/` never calls a language model, reads files or the clock, or generates ids. That is checked by `grep` and tests, and it is why results are reproducible.

10. Configuration and data

10.1 `data/config.yaml`

Every rate, speed, weight, threshold and message template lives here. Each entry has a `value`, a `source_status` (`sourced` or `assumed`) and a `note`. There are 67 entries: 4 sourced (the IST offset, the road and water CO_2 factors, and an IBM model id) and 63 assumed. The two CO_2 factors cite IPCC AR6 WG3 Table 10.1 and have **not** been re-checked against it by the team.

To change a value: edit it in `config.yaml` and restart the backend (the file is read once at start-up). Keep the tag and note. Then run `python docs/sync_tables.py` so the documentation tables match, and re-run the tests.

10.2 `data/seed.json`

Fictional demo data: 4 jetties (illustrative names, approximate positions), 7 vessels, 2 operators, 2 sailings, cargo, shippers, 2 warehouses, 31 delivery records and 2 disruption scenarios. All people, vessels and history are tagged `simulated`. Jetty names and coordinates are illustrative and should be verified with IWAI or the Kerala State Water Transport Department. *Reset Demo* restores this file's contents.

10.3 Environment variables

Variable	Default	Meaning
`JALSETU_MOCK_LLM`	`1`	1 = offline pattern reader (used for the demo). 0 = try the optional IBM watsonx.ai adapter (never run in this project).
`WATSONX_API_KEY`, `WATSONX_PROJECT_ID`, `WATSONX_URL`	unset	Only read when `JALSETU_MOCK_LLM=0`. Dummy values are in `.env.example`; never commit real keys.
`VITE_API_URL` (frontend)	`http://localhost:8000/api`	Backend address used by the web app. Put it in `frontend/.env`.

11. API reference

All paths start with `/api`. Full, interactive documentation: `http://localhost:8000/docs`. Contracts for features 004 to 006 are in `specs/*/contracts/openapi.yaml`.

Method	Path	Purpose
POST	/options	Compare road and hybrid plans; recommendation and reason
POST	/match	Rank sailings for a cargo (hazard filter; excluded list)
POST, GET	/cargo, /cargo/list	Create and list cargo
POST	/book, /book/{id}/confirm	Preview a booking; explicit confirmation
POST	/compliance/review/{id}/approve	Simulated compliance review for hazardous cargo
POST	/parse, /operator/message	Free-text intake (offline parser)
POST, GET	/vessel/register, /vessel/list, /vessel/parse-message	Vessel registration and message reading
POST	/sailing/publish, GET /sailing/list	Publish and list sailings
POST	/admin/vessel/{id}/verify, /suspend, /reinstate	Demo admin actions (simulated, no authentication)
POST, GET	/booking/{id}/deliver, /operator/{id}/profile	Record a delivery; on-time record and history
GET	/bookings, /booking/{id}	Shipments with milestones and event log
POST	/demo/booking/{id}/advance, /disrupt	Demo controls: next milestone; simulated disruption
POST	/replan/{id}, /replan/{id}/confirm	Re-plan proposal (read-only); explicit confirmation
POST	/booking/{id}/notify	Notification preview (never sent)
GET	/disruptions/summary, /viability	Planner analytics
GET, POST	/warehouse/list, /register, /stage, /stage/confirm, /utilisation	Warehouses and staging

GET	/nodes, /operators, /hazard-classes, /config/assumptions	Reference data; all config values with tags
POST	/demo/reset	Restore seed data

Example request

```
curl -X POST http://localhost:8000/api/options -H "Content-Type: application/json" \
  -d '{"weight_t":10,"origin_node_id":"node-001","destination_node_id":"node-002",
       "ready_time":"04:00","deadline_h":10}'
# expected: "recommended":"hybrid"   (use "deadline_h":3 to get "road")
```

Errors come back as JSON. Field errors look like `{"detail":{"errors":[{"field":"draft_m","code":"not_positive","message":"..."}]}}`; state conflicts look like `{"detail":{"error":"invalid_transition"}}`.

12. Optional tasks

12.1 Regenerate the screenshots

Needs Google Chrome or Microsoft Edge, and one extra package that is not in `requirements.txt`. With both servers running:

```
pip install websockets
python docs/screenshots/capture.py
```

This drives a headless browser through the walkthrough and writes twelve images to `docs/screenshots/`. It resets the demo state before and after.

12.2 Refresh the documentation tables

```
python docs/sync_tables.py
```

Rewrites the generated tables in `docs/SPEC.md` from the current config and seed. The test `test_docs_sync.py` fails if they are stale.

12.3 IBM watsonx.ai adapter (not exercised)

An adapter for IBM watsonx.ai Granite exists in `backend/api/llm/watsonx.py` for optional text parsing. It has **never been run** in this project and is not needed. If you want to try it: install `ibm-watsonx-ai`, set the three `WATSONX_*` variables in your shell, and start the backend with `JALSETU_MOCK_LLM=0`. If initialisation fails the backend falls back to the offline reader. Even then, the language model only turns text into fields; it never prices, ranks or recommends.

13. Troubleshooting

Symptom	Likely cause	Fix
pip: "No matching distribution found for rapidfuzz==3.14.6"	Corrupt or stale pip cache	`pip install --no-cache-dir -r backend/requirements.txt`

PowerShell: "running scripts is disabled"	Execution policy	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again
`No module named 'backend'`	Started from the wrong folder	`cd` to the repository root and re-run the uvicorn command
App says "Cannot reach the JalSetu server"	Backend not running, wrong port, or frontend not on 5173	Check `http://localhost:8000/`; keep backend on 8000 and frontend on 5173 (section 5)
"Address already in use"	Another program uses the port	Close it, or stop the previous backend / Vite process. Changing ports needs the CORS and `VITE_API_URL` changes in section 5
`npm install` or Vite errors	Node version too old	Install a recent Node (tested 24.14.0) and re-run `npm install`
A code or config edit does not appear	Backend reads config once; Vite may serve a stale file	Restart the backend after config changes; restart `npm run dev` after frontend changes
Tests fail after editing config or seed	Docs tables out of date	`python docs/sync_tables.py`, then re-run the tests
Data looks wrong or leftover from a previous run	In-memory state persists until reset	Click **Reset Demo** or restart the backend
"Confirm booking" is disabled	Hazardous cargo awaits compliance review	Record the review (demo button) first; this is intended behaviour
HTTP 409 errors in the UI, e.g. "already delivered"	Repeated action on the same record	Intended guard against double actions; reset the demo to start over

14. Limitations and what is real

Label	Meaning
BUILT AND TESTED	Implemented and covered by passing automated tests
BUILT, MOCKED	Runs, but data or actors are simulated and tagged `[simulated]`
ROADMAP	Not built, or code exists but has never been run

- **Built and tested:** road vs hybrid decisions; sailing matching and exclusions; two-step booking; hazard filter; vessel and sailing registration; reliability record; disruption re-plan with explicit confirm; warehouse staging; planner analytics.
- **Built, mocked:** vessel verification, compliance reviewer, milestones, disruptions, notification previews, delivery history, warehouse and operator data, the plain-language chat.
- **Roadmap:** live GPS and AIS tracking, real alerts, payments, authentication, vessel-registry checks, routed road distances and chart-based waterway distances, Malayalam intake, a database, live IBM watsonx.ai parsing.

Modelling limits to keep in mind

- Distances are straight-line multiplied by a factor, not routed or charted. Only four illustrative jetties exist.
- The hybrid plan's cost and CO_2 exclude first and last-mile trucking (see the Step 1 caveat).
- Hazard rules are assumed placeholders, not regulations. The system gives eligibility guidance only and refers users to IWAI, PESO or the state authority.
- The monsoon reliability penalty exists but is not switched on; sailing weekdays are stored but not used by the engine; the schedule part of the match score is a fixed neutral value.
- Re-plans use a demo clock (the planned time of the current milestone), treat earlier payments as sunk, and do not change sailing capacity.
- No authentication: the verify, suspend, compliance-review, advance and disrupt endpoints are open demo controls. State is in one process and in memory.
- The web app has no automated tests; it was checked in headless Chrome at 1280 px only. No mobile layout or formal accessibility audit.

15. Third-party tools, AI tools and licences

Category	Items
Python libraries	FastAPI, Starlette, Uvicorn, Pydantic, PyYAML, RapidFuzz, httpx, python-dotenv, pytest
JavaScript libraries	React, React DOM, Vite, @vitejs/plugin-react, oxlint, @types/react, @types/react-dom
Dev-only tooling	`websockets` (Python) and a local headless Chrome or Edge, used only to regenerate screenshots
AI coding tools	**IBM Bob**, IBM's AI coding agent and our development partner throughout (code generation, spec writing, task implementation); Claude Code by Anthropic (features 004 and 005, the role-based UI, cleanup and documentation)
Specification tooling	GitHub Spec Kit structure and workflow (constitution, specify, clarify, plan, tasks, analyze, implement). For features 004 and 005 the steps were run by hand rather than through its command line
LLM provider	IBM watsonx.ai Granite 13B chat (optional adapter, **not used** in the demo)

Exact versions, licences and URLs are in `ATTRIBUTIONS.md`; the full AI-tool disclosure is in `docs/disclosures.md`. No real personal data is stored or sent anywhere; all demo people, vessels and contacts are fictional, and phone numbers and emails are removed before any text reaches a parser.

Appendix A. Command cheat sheet and tested environment

Every command in order (Git Bash)

```
# one-time setup
git clone https://github.com/gop-i-krishnan/keralai-team-vortex.git
cd keralai-team-vortex
python -m venv .venv
source .venv/Scripts/activate
pip install -r backend/requirements.txt
(cd frontend && npm install)

# every time: terminal 1 (repository root, venv active)
JALSETU_MOCK_LLM=1 python -m uvicorn backend.api.main:app --port 8000

# every time: terminal 2
cd frontend && npm run dev            # then open http://localhost:5173

# checks
python -m pytest backend/ -q            # expect: 261 passed
(cd frontend && npm run build)       # expect: build succeeds
```

Same, in Windows PowerShell

```
# one-time setup
git clone https://github.com/gop-i-krishnan/keralai-team-vortex.git
cd keralai-team-vortex
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend/requirements.txt
cd frontend; npm install; cd ..

# terminal 1
$env:JALSETU_MOCK_LLM="1"; python -m uvicorn backend.api.main:app --port 8000

# terminal 2
cd frontend; npm run dev
```

Tested environment

Item	Value
Operating system	Windows 11 Home
Python / pip	3.14.5 / 26.1.1
Node.js	24.14.0
Backend packages	fastapi 0.142.2, starlette 1.7.0, uvicorn 0.54.0, pydantic 2.13.5, pyyaml 6.0.3, rapidfuzz 3.14.6, pytest 9.1.1, httpx 0.28.1, python-dotenv 1.2.4
Frontend packages	react 19.3.0, react-dom 19.3.0, vite 8.3.3, @vitejs/plugin-react 6.1.2, oxlint 1.87.0
Checks (7 Oct 2026)	261 tests passed (also with non-loopback network access blocked); fresh clone from GitHub, install, build, both servers and a full browser walkthrough passed